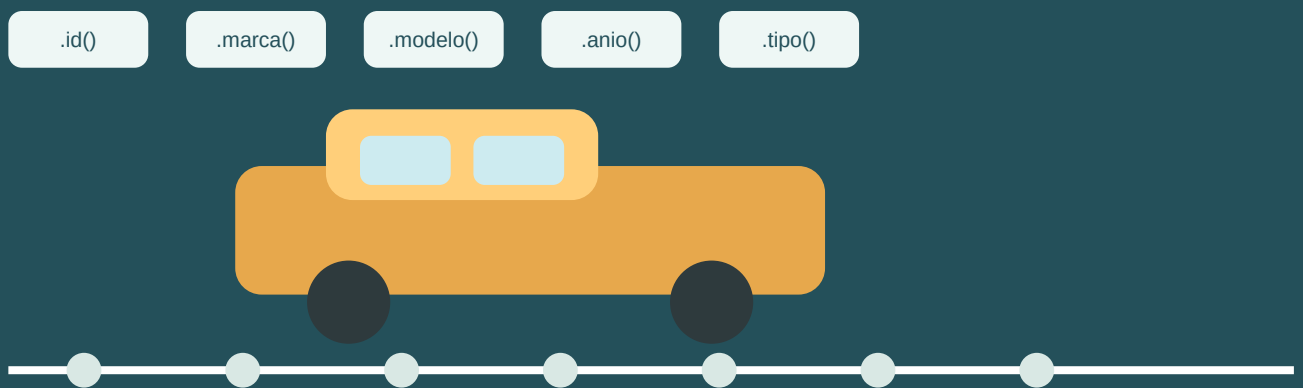


Patron Builder

Guia visual para entender que significa, cuando usarlo y como aplicarlo con Lombok en Java/Spring.



Ejemplo base: proyecto patron-builder, endpoint `/v1/builder/carrofull`, clases `Carro` y `TipoCarro`.

1. La Idea En Una Frase

Builder es un patron creacional: ayuda a construir objetos complejos paso a paso, con una API clara, evitando constructores gigantes y estados a medio armar. En Java moderno, Lombok suele generar el builder por nosotros con **@Builder**.

Analogia rapida: pedir un carro personalizado. No dices “dame este objeto con 14 parametros en este orden exacto”; eliges marca, modelo, año, tipo de transmision y al final el taller entrega el carro terminado. Ese “taller” es el builder.

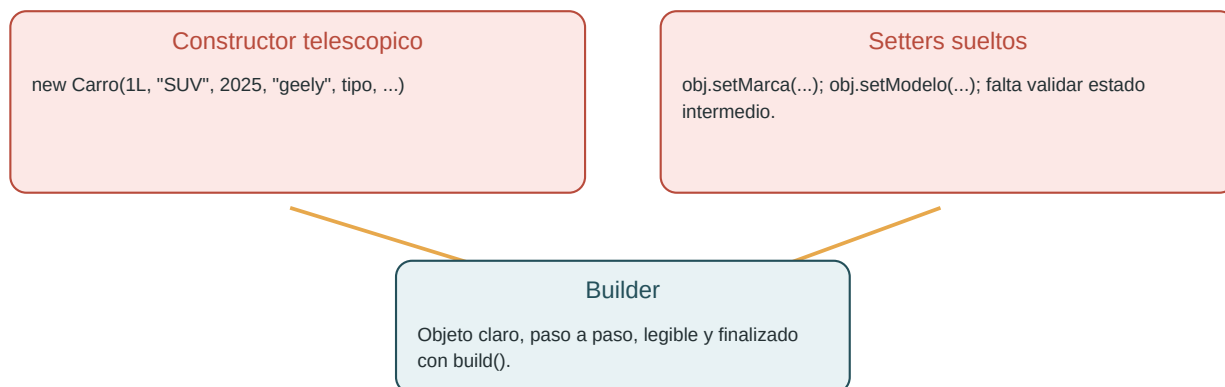


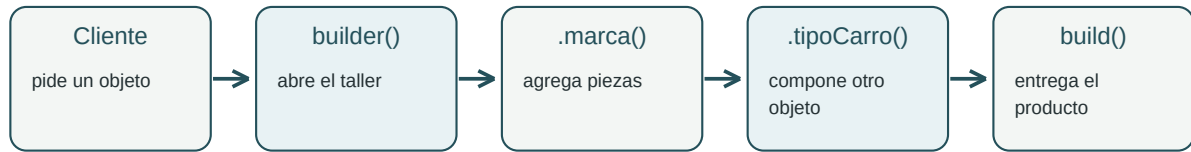
Imagen 1. Builder aparece cuando crear un objeto empieza a ser ruidoso, fragil o difícil de leer.

2. El Problema Que Resuelve

- **Constructores telescópicos:** muchos parametros, varios opcionales y errores faciles por orden incorrecto.
- **Setters sueltos:** el objeto puede existir en estados incompletos antes de terminar de configurarse.
- **Legibilidad:** `Carro.builder().marca("geely").anio(2025)` se entiende mejor que un constructor largo.
- **Composicion:** puedes construir objetos anidados, como `TipoCarro.builder()`, dentro del objeto principal.

El beneficio principal no es “usar un patron porque si”; es separar la configuracion paso a paso del resultado final.

3. Flujo Mental



Eliges piezas, encadenas pasos y cierras con build().

Imagen 2. El flujo tipico en Lombok: llamar builder(), encadenar atributos y cerrar con build().

```

Carro carro = Carro.builder()
    .id(1L)
    .marca("geely")
    .modelo("SUV")
    .anio(2025)
    .tipoCarro(TipoCarro.builder()
        .id(2L)
        .tipo("AUTOMATICO")
        .build())
    .build();
  
```

4. Roles Del Patron Clasico

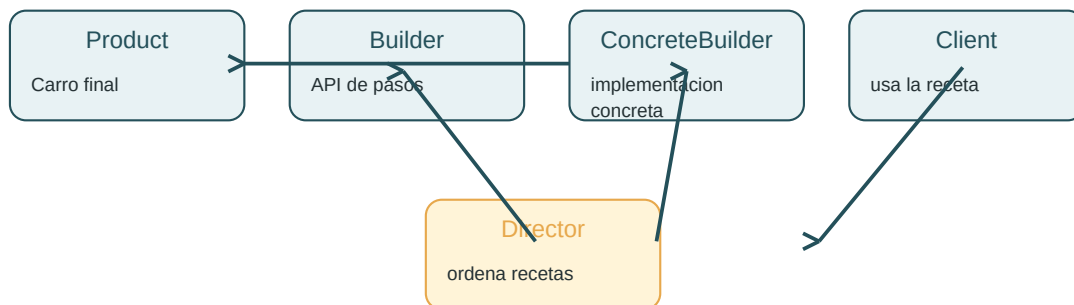


Imagen 3. En el patron clasico hay roles separados. Con Lombok, varios roles se vuelven codigo generado y quedan escondidos detras de la anotacion.

- **Product:** el objeto final, por ejemplo Carro.
- **Builder:** la interfaz o clase con pasos de construccion.
- **ConcreteBuilder:** implementacion que va guardando valores.
- **Director:** opcional; define recetas. En tu ejemplo no hace falta.
- **Client:** controller, service o test que llama al builder.

5. Builder Manual Vs Lombok

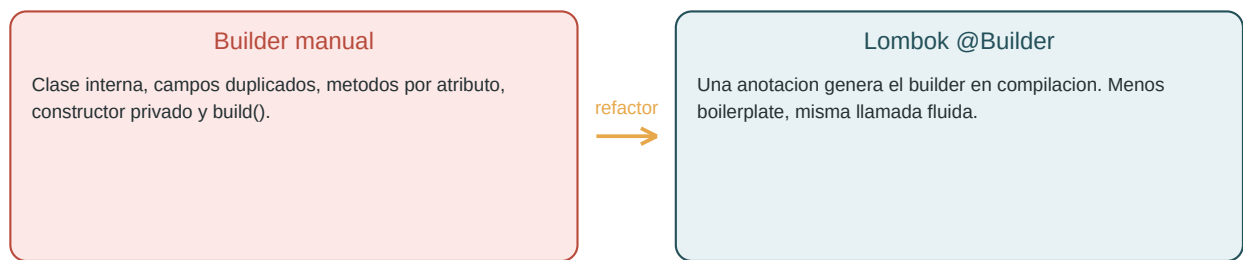


Imagen 4. El builder manual de Libro repetia estructura que Lombok ya sabe generar.

Builder manual	Lombok @Builder
Mucho codigo repetitivo: clase interna, campos duplicados, metodos por atributo, constructor privado.	Declaras campos y anotaciones. Lombok genera el builder en compilacion.
Mas control artesanal, pero mas mantenimiento.	Menos ruido y estilo habitual en proyectos Java modernos.
Puede ser util si necesitas reglas muy especiales.	Ideal para DTOs, request/response models, objetos de dominio simples y ejemplos didacticos.

6. Aplicado A Tu Proyecto

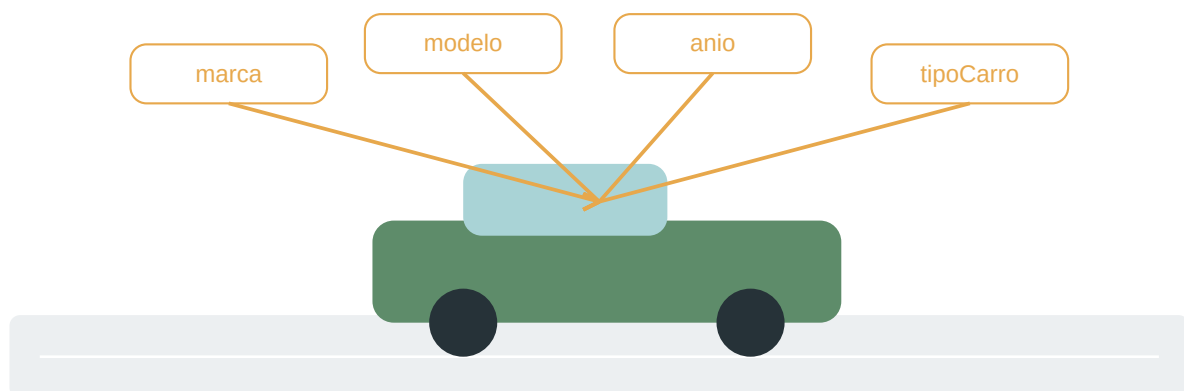


Imagen 5. En el proyecto ahora todo el ejemplo gira alrededor de Carro y su composicion con TipoCarro.

```
@Builder
@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
public class Carro {
    private Long id;
    private String modelo;
    private Integer anio;
    private String marca;
    private TipoCarro tipoCarro;
}
```

El controller queda como cliente del builder: no conoce una clase interna manual, solo consume la API fluida que Lombok genera.

7. Cuando Usarlo

- Cuando un objeto tiene varios campos opcionales o combinaciones de configuracion.
- Cuando el constructor empieza a ser dificil de leer o recordar.
- Cuando quieres construir objetos anidados de forma expresiva.
- Cuando estas creando DTOs o modelos de respuesta en tests y ejemplos.
- Cuando quieres que el codigo lector entienda el nombre de cada valor.

No es obligatorio para objetos diminutos con dos campos y un constructor claro. Un patron ayuda cuando reduce confusion; si agrega ceremonia sin necesidad, deja de ayudar.

8. Cuando Evitarlo

- Si el objeto siempre se crea con todos sus campos y el constructor es corto.
- Si necesitas invariantes fuertes y el builder no las valida antes de `build()`.
- Si usar setters despues de construir el objeto rompe reglas del dominio. En ese caso considera objetos inmutables.
- Si el equipo no usa Lombok o el proyecto no tiene annotation processing bien configurado.

9. Buenas Practicas Con Lombok

- Usa `@Builder` para evitar escribir a mano builders simples.
- Si hay valores por defecto, usa `@Builder.Default` y asigna el valor en el campo.
- Para listas o colecciones, revisa `@Singular` cuando quieras agregar un item por llamada.
- Si vas a deserializar JSON hacia builders, revisa `@Jacksonized`.
- Si el dominio exige reglas, valida antes de entregar el objeto: constructor, metodo `factory` o builder personalizado.
- Si quieres inmutabilidad, combina la idea con campos `final`, getters y sin setters.

10. Mini Receta Para Decidir

Pregunta	Si la respuesta es si...
¿Hay muchos parametros?	Builder mejora legibilidad.
¿Hay parametros opcionales?	Builder evita constructores sobrecargados.
¿Hay objetos anidados?	Builder permite componer paso a paso.
¿El objeto debe ser siempre valido?	Agrega validacion en el punto de construccion.
¿Solo hay dos campos simples?	Constructor o static factory puede bastar.

11. Ejercicio Mental

Mira esta llamada y preguntate: ¿puedo entender el objeto sin ir a ver la firma del constructor?

```
Carro.builder()  
    .marca("Toyota")  
    .modelo("Corolla")  
    .anio(2026)  
    .tipoCarro(TipoCarro.builder().tipo("HIBRIDO").build())  
    .build();
```

Si la respuesta es si, el builder esta cumpliendo su trabajo: volver explicita la intencion del codigo.

12. Referencias Para Profundizar

Project Lombok - @Builder

<https://projectlombok.org/features/Builder>

Documentacion oficial de Lombok sobre lo que genera @Builder, @Builder.Default y @Singular.

Refactoring.Guru - Builder

<https://refactoring.guru/design-patterns/builder>

Explicacion visual del patron, roles clasicos y problemas que resuelve.

Refactoring.Guru - Builder en Java

<https://refactoring.guru/design-patterns/builder/java/example>

Ejemplo en Java usando carros y manuales, muy cercano al caso de estudio.

SourceMaking - Builder Design Pattern

https://sourcemaking.com/design_patterns/builder

Resumen clasico del intento del patron, estructura y checklist.

Design Patterns: Elements of Reusable Object-Oriented Software

Gamma, Helm, Johnson y Vlissides, 1994

Libro original de la Gang of Four donde se formalizaron estos patrones.

Effective Java

Joshua Bloch, Item 2

Recomienda builders cuando hay muchos parametros de constructor, especialmente opcionales.

Cierre

Builder no es magia: es una forma ordenada de construir. Lombok tampoco cambia la idea; simplemente evita que escribas a mano el codigo repetitivo. En tu proyecto, eso significa dejar el ejemplo en `Carro.builder()` y borrar el builder manual de Libro.